

CSE326: Internet Programming & Web Technologies

Master Handwritten-Style 1-Shot Revision Notes

Comprehensive Unit 1 to Unit 6 Quick Review, Syntaxes, Tables & High-Yield Alerts

School of Computer Science & Engineering
Lovely Professional University

Academic Year 2024–2025 / 2025–2026

Contents

1	Unit 1: HTML5 Fundamentals, Document Architecture & Protocols	2
2	Unit 2: Semantic HTML5, Tables, Multimedia & Forms Architecture	4
3	Unit 3: CSS3 Mastery, Selectors, Typography & The Box Model	6
4	Unit 4: JavaScript Core Mechanics, Variables, Data Types & Control Flow	8
5	Unit 5: JavaScript Functions, DOM, Events & Regex Form Validation	10
6	Unit 6: Web Application Architecture, Debugging, Web Storage & Deployment	12

1 Unit 1: HTML5 Fundamentals, Document Architecture & Protocols

Core Concept 1.1: Web vs. Internet & TCP/IP Stack

- **Internet (1983, TCP/IP):** Global interconnected network of hardware and networks.
- **World Wide Web (1989/1990, Tim Berners-Lee):** Application-layer service using HTTP to transfer hypertext documents.
- **TCP/IP 4-Layer Architecture:**
 1. **Application:** HTTP (Port 80), HTTPS (Port 443), DNS (Port 53), SMTP (Port 25).
 2. **Transport:** TCP (Connection-oriented, 3-way handshake: SYN → SYN-ACK → ACK, reliable), UDP (Connectionless, fast).
 3. **Internet:** IP (IPv4: 32-bit dotted-decimal, IPv6: 128-bit hex), Routing.
 4. **Link:** Ethernet, Wi-Fi (802.11), MAC addressing.

URL Anatomy & DNS Resolution

URL Structure: `https://www.example.com:443/courses/web.html?id=101#unit1`

- **Scheme:** `https://` | **Host:** `www.example.com` | **Port:** `443` | **Path:** `/courses/web.html` | **Query:** `?id=101` | **Fragment:** `#unit1`
- **DNS:** Resolves human-readable domain names (`example.com`) to IP addresses (`93.184.216.34`).

Essential HTML5 Document Template

```
<!DOCTYPE html> <!-- Standards Mode Trigger -->
<html lang="en">
<head>
  <meta charset="UTF-8"> <!-- Character encoding: supports symbols e.g.
    INR, EUR, Emojis -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0"> <
    !-- Mobile responsiveness -->
  <meta name="description" content="SEO snippet">
  <title>Document Title</title>
</head>
<body>
  <!-- Visible Page Content -->
</body>
</html>
```

Core Concept 1.2: Semantic vs. Presentational Elements

- **Headings:** `<h1>` (Largest, single primary title) down to `<h6>` (Smallest sub-heading).
- **Strong vs. Bold:** `<strong>` = Semantic importance/urgency (vocalized with stress by screen readers); `<b>` = Stylistic bold font only.
- **Emphasize vs. Italic:** `<em>` = Semantic emphasis; `<i>` = Stylistic italic/alternate voice.
- **Definitions vs. Citations:** `<dfn>` = Defining instance of a term; `<cite>` = Title of a creative work (book, film, paper, song).

- **Sub/Superscript:** `<sub>` (Subscript: H₂O) vs. `<sup>` (Superscript: 2⁵ = 32).
- **Special Formatting:** `<mark>` (Yellow highlight), `<small>` (Fine print), `<abbr title="World Health Organization">WHO</abbr>`, `<hr>` (Thematic divider, void tag), `<br>` (Line break, void tag).

Block-level vs. Inline Elements

- **Block-level:** Starts on a new line, takes 100% available width (`<div>`, `<p>`, `<h1>`-`<h6>`, `<ul>`, `<ol>`, `<section>`, `<article>`, `<header>`, `<footer>`, `<main>`).
- **Inline:** Flows inside text line, takes only required width (`<span>`, `<a>`, `<img>`, `<strong>`, `<em>`, `<b>`, `<i>`, `<label>`).

2 Unit 2: Semantic HTML5, Tables, Multimedia & Forms Architecture

Core Concept 2.1: Semantic Structural Landmarks

- `<header>`: Introductory content and navigation for page or `<article>`.
- `<nav>`: Container for major navigation links.
- `<main>`: Dominant central content of document (strictly **one** per page).
- `<article>`: Self-contained, independently distributable syndication content (e.g. blog post, product card).
- `<section>`: Thematic grouping of content, typically with its own heading.
- `<aside>`: Tangential sidebars, quotes, or adverts.
- `<footer>`: Author, copyright (©), and legal links.
- `<figure>` & `<figcaption>`: Media container with caption binding.

HTML Lists & Hyperlinks

- **Unordered List:** `<ul style="list-style-type: disc|circle|square|none;">`
`<li>...</li>` `</ul>`
- **Ordered List:** `<ol type="1|a|A|i|I" start="3" reversed>` `<li>...</li>` `</ol>`
- **Description List:** `<dl>` `<dt>Term</dt>` `<dd>Definition</dd>` `</dl>`
- **Anchor Tag:** `<a href="URL" target="_blank" rel="noopener noreferrer">Text</a>`
 - `mailto:user@domain.com` (Email link) | `tel:+919876543210` (Phone call link)
 - `#sectionID` (Internal page fragment bookmark jump)
- **Image Tag:** ``

Core Concept 2.2: Tables & Spanning

```
<table border="1" style="border-collapse: collapse;">
  <caption>Student Fee Matrix</caption>
  <thead>
    <tr>
      <th scope="col" rowspan="2">Roll No</th>
      <th scope="col" colspan="2">Fees (INR)</th>
    </tr>
    <tr>
      <th scope="col">Tuition</th>
      <th scope="col">Hostel</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>101</td>
      <td>80,000</td>
      <td>45,000</td>
    </tr>
  </tbody>
</table>
```

```
</tr>
</tbody>
</table>
```

- `colspan="N"`: Merges N columns horizontally.
- `rowspan="N"`: Merges N rows vertically.
- `scope="col|row"`: Screen reader accessible column/row association.

Core Concept 2.3: Forms Architecture & Input Controls

- **Form Attributes:** `action="submit.php" | method="GET"` (URL query string, non-sensitive) vs. `method="POST"` (HTTP body, secure, file uploads) | `enctype="multipart/form-data"` (Mandatory for file uploads).
- **Form Controls:**
 - Text: `type="text", type="password", type="email", type="number" (min, max, step), type="tel" (pattern)`.
 - Selection: `type="radio" (Grouped via identical name), type="checkbox"`.
 - Dropdown: `<select name="city" id="c" multiple size="4"> <option value="del" selected>Delhi</option> </select>`
 - Multi-line: `<textarea name="msg" rows="4" cols="50" placeholder="..."></textarea>`
 - File Upload: `<input type="file" accept=".pdf,.png" multiple>`
- **Accessibility Rule:** Every control must bind to an explicit `<label for="id">`.

3 Unit 3: CSS3 Mastery, Selectors, Typography & The Box Model

Core Concept 3.1: CSS Inclusion & The Cascade

1. **Inline CSS:** `<h1 style="color: blue; text-align: center;">`
2. **Internal CSS:** `<style> h1 color: blue; </style>` inside `<head>`.
3. **External CSS:** `<link rel="stylesheet" type="text/css" href="style.css">`
4. **The Cascade Algorithm:** Importance (!important) → Specificity → Source Order (Last rule wins in a tie).

CSS Selectors & Specificity Calculation

Specificity Weight Triple (a, b, c) :

- **a (IDs):** `#main-title` → $(1, 0, 0)$
- **b (Classes, Attributes, Pseudo-classes):** `.card`, `[type="text"]`, `:hover`, `:nth-child()` → $(0, 1, 0)$
- **c (Type/Element, Pseudo-elements):** `div`, `p`, `::before`, `::after` → $(0, 0, 1)$
- Universal selector `*`, combinators `+`, `>`, `~`, `space` contribute $(0, 0, 0)$.

Combinators:

- **Descendant** (`div p`): Any `<p>` inside `<div>`.
- **Child** (`div > p`): Direct child `<p>` only.
- **Adjacent Sibling** (`h2 + p`): First `<p>` immediately following `<h2>`.
- **General Sibling** (`h2 ~ p`): All sibling `<p>` elements following `<h2>`.

Core Concept 3.2: The CSS Box Model

MARGIN (Transparent space outside the border)
BORDER (Stroke line surrounding padding: width, style, color)
PADDING (Clear space inside border around content)
CONTENT (Dimensions: width × height)

- **Default box-sizing:** `content-box`:

Rendered Box Width = width + padding-left/right + border-left/right

- **Universal Reset box-sizing:** `border-box`:

Rendered Box Width = width (padding and border are absorbed inside width)

- **Margin Collapsing:** Adjoining vertical margins in normal flow collapse to the larger single margin value.

Backgrounds & Typography Properties

- **Shorthand Background:** `background: url("bg.jpg") no-repeat center center / cover fixed #f0f0f0;`
 - `background-size: cover` (Fills container, crops excess) vs. `contain` (Full image visible).
 - `background-attachment: fixed` (Parallax viewport-pinned effect).
- **Text Properties:** `color, text-align: center|justify, text-decoration: none|underline, text-transform: uppercase|capitalize, letter-spacing: 2px, word-spacing: 5px, line-height: 1.6.`

4 Unit 4: JavaScript Core Mechanics, Variables, Data Types & Control Flow

Core Concept 4.1: Variables & Scoping Comparison

Keyword	Scope	Re-declare	Re-assign	Hoisted / TDZ
var	Function	✓ Yes	✓ Yes	Hoisted (undefined), No TDZ
let	Block {}	× No	✓ Yes	Hoisted (Uninitialized), TDZ
const	Block {}	× No	× No	Hoisted (Uninitialized), TDZ

Note: `const arr = [1, 2]; arr.push(3);` is **valid** because the object reference is constant, while contents remain mutable.

Data Types & Equality Rules

- **7 Primitive Types** (Passed by value, immutable): `number`, `string`, `boolean`, `undefined`, `null`, `symbol`, `bigint`.
- **Reference Types** (Passed by pointer, mutable): `Object`, `Array`, `Function`.
- `typeof null === "object"` (Legacy engine bug).
- `typeof [] === "object" → Use Array.isArray([]) (returns true).`
- **Equality:** `5 == "5"` is `true` (Coercion) vs. `5 === "5"` is `false` (Strict).

Core Concept 4.2: Operators & Expressions

- **Arithmetic:** `+` (Addition / String concat), `-`, `*`, `/`, `%` (Modulus remainder), `**` (Power).
- **String Concatenation:** `"10" + 20 → "1020"` | `"20" - 5 → 15`.
- **Logical Operators:** `&&` (AND), `||` (OR), `!` (NOT).
- **Nullish Coalescing (`??`):** Returns right operand only if left is `null` or `undefined`.

Core Concept 4.3: Control Flow, Loops & Popups

- **Switch Statement:** Uses strict equality (`===`). Without `break;`, execution **falls through** all subsequent cases.
- **Loop Types:**
 - `while (condition)`: Pre-tested (0 or more executions).
 - `do { ... } while (condition)`: Post-tested (**Guaranteed at least 1 execution**).
 - `for (let i=0; i<N; i++)`: Counted loop.
 - `break` (Terminates loop immediately) vs. `continue` (Skips current iteration).
- **Browser Popups:**

- `alert("msg")`: Informational modal (returns `undefined`).
- `confirm("Are you sure?")`: Yes/No prompt (returns `true` / `false`).
- `prompt("Enter Name:", "Default")`: Input prompt (returns `string` or `null`).

5 Unit 5: JavaScript Functions, DOM, Events & Regex Form Validation

Core Concept 5.1: Functions & Methods

- **Named Function (Hoisted):** `function add(a, b) { return a + b; }`
- **Function Expression (Not Hoisted):** `const add = function(a, b) { return a + b; };`
- **Arrow Function (ES6):** `const square = x => x * x;` (Lexical this, no arguments).
- **Default Parameter:** `function greet(name = "Guest") { return `Hello ${name}`; }`
- **Object Methods & this:**

```
let student = {
  name: "Alice",
  age: 21,
  details: function() { return `_${this.name}_ is _${this.age}_`; }
};
```

DOM Tree Selection & Manipulation

- **Selectors:**
 - `document.getElementById("id")` → Single Node.
 - `document.getElementsByClassName("cls")` → Live HTMLCollection.
 - `document.querySelector(".box > p")` → First matching Node.
 - `document.querySelectorAll(".box > p")` → Static NodeList.
- **Modifying Content & Styles:**
 - `el.innerHTML = "<b>Bold</b>"` (Parses markup) vs. `el.textContent = "Text"` (Safe text).
 - `el.style.color = "red" | el.style.backgroundColor = "yellow"`
 - `el.classList.add("active") | el.classList.remove("active") | toggle()`
 - `el.setAttribute("src", "new.jpg") | el.getAttribute("src")`
- **Node Creation:** `let p = document.createElement("p"); p.textContent = "New"; document.body.appendChild(p);`

Core Concept 5.2: Event Handling

- **Standard Listener:** `btn.addEventListener("click", function(event) { ... });`
- **Common Events:**
 - **Mouse:** `click`, `dblclick`, `mouseover`, `mouseout`.
 - **Keyboard:** `keydown`, `keyup` (Inspect `event.key`).
 - **Form:** `submit`, `change`, `focus`, `blur`, `input`.
- `event.preventDefault()`: Halts native browser actions (e.g. cancels invalid form submission).
- `event.stopPropagation()`: Stops event from bubbling up parent nodes.

Core Concept 5.3: Form Validation & Regular Expressions (Regex)

- **Form Submission Hook:** `<form onsubmit="return validateForm()">` (Returning false aborts submission).

- **Regex Pattern Anatomy:**

$$/^[\^]+@[\^]+\.[a-z]{2,3}\$/$$

- `^` : Start of string anchor.
- `[^ ]+` : 1 or more non-space characters.
- `@` : Literal @ symbol.
- `\.` : Literal dot character (escaped).
- `[a-z]{2,3}` : 2 to 3 lowercase alphabets (.com, .in, .org).
- `$` : End of string anchor.

- **Test Method:** `pattern.test(emailString)` → Returns boolean true/false.

6 Unit 6: Web Application Architecture, Debugging, Web Storage & Deployment

Core Concept 6.1: Browser Developer Tools (F12)

- **Elements Tab:** Live DOM inspection, on-the-fly CSS editing, Box Model metrics verification.
- **Console Tab:** JavaScript REPL, logging (`console.log`, `console.table`, `console.error`).
- **Sources Tab:** Set breakpoints, step over (F10), step into (F11), call stack inspection, debugger; statement.
- **Network Tab:** Inspect HTTP requests, asset waterfall timing, status codes (200, 404), Slow 3G throttling.
- **Application Tab:** Inspect `localStorage`, `sessionStorage`, cookies, and cache.

Core Concept 6.2: Web Storage API (Local vs. Session Storage vs. Cookies)

Property	localStorage	sessionStorage	Cookies
Capacity	~ 5 MB – 10 MB	~ 5 MB	~ 4 KB
Lifespan	Persistent until cleared	Tab/Session lifetime	Expiration date set
Server Sent	Never sent with HTTP	Never sent with HTTP	Sent on every request
Syntax	<code>localStorage.setItem("key", "value")</code>	<code>sessionStorage.setItem("key", "value")</code>	<code>document.cookie="..."</code>

Object Storage: Always serialize before storing: `localStorage.setItem("user", JSON.stringify(obj))`; and parse on retrieval: `JSON.parse(localStorage.getItem("user"))`;

Core Concept 6.3: Version Control (Git) & GitHub Pages Deployment

• Core Git Commands:

```
git init                    # Initialize local git repository
git add .                  # Stage all modified files
git commit -m "feat: initial commit" # Create immutable snapshot
git remote add origin <URL> # Link remote GitHub repo
git push -u origin main    # Push to remote branch
```

• GitHub Pages Static Hosting:

- Free static hosting served directly over HTTPS from repository branch `main` (root or `/docs`).
- Serves `index.html` by default.
- Must use **relative asset paths** (`./css/style.css` instead of `/css/style.css`).

Performance Best Practices & Core Web Vitals

- **Critical Rendering Path:** HTML → DOM, CSS → CSSOM → Render Tree → Layout → Paint → Composite.
- **Script Loading:** Use `<script src="..." defer>` to prevent blocking HTML parser.
- **Image Optimization:** Use modern formats (WebP), specify `width/height` to prevent Cumulative Layout Shift (CLS).

tive Layout Shift (CLS), add `loading="lazy"`.

- **Core Web Vitals:** LCP (Largest Contentful Paint $\leq 2.5s$), INP (Interaction to Next Paint $\leq 200ms$), CLS (Cumulative Layout Shift ≤ 0.1).